



Sistem de procesare și calculare a mediei pe 8 biți

Studenti:
Sîrb Bogdan Cristian
Stănescu Paul
Grupa 30217

Coordonator:
Prof. Spătariu Diana
Prof. Văcariu Lucia

Mai 2011

CUPRINS

1.SPECIFICAȚIA PROIECTULUI	4
2.SCHEMA BLOC	4
3.PROIECTARE	5
4.INTRĂRI ȘI IEȘIRI. SEMNALE INTERNE	14
5.JUSTIFICAREA SOLUȚIEI ALESE	16
6.INSTRUCȚIUNI DE UTILIZARE	17
7.POSIBILITĂȚI DE DEZVOLTARE ULTERIOARĂ. IMBUNĂTĂȚIRI	19

1.SPECIFICAȚIA PROIECTULUI:

CERINȚĂ: Dezvoltarea unui sistem simplu de prelucrare a unui semnal paralel pe 8 biți care va calcula media. Design-ul va fi implementat pe Xilinx/Digilent Spartan 3 XC3S200. Sistemul va fi dezvoltat în VHDL utilizând modelul Xilinx ISE WebPack Version 6.3.

SPECIFICAȚII: Va fi necesar un sistem de filtrare pentru a permite sistemul de calcul să realizeze operațiile în timp real, atât afișarea numerelor generate aleator cât și media efectuată până în acel moment. Sarcina este de a dezvolta în VHDL sistemul de calcul al mediei combinat cu un divizor de frecvență la care se adauga folosirea butoanelor și ale switch-urilor cât și a display-ului pe 7 segmente. Specificatia switch-urilor va fi următoarea:

Panoul de control

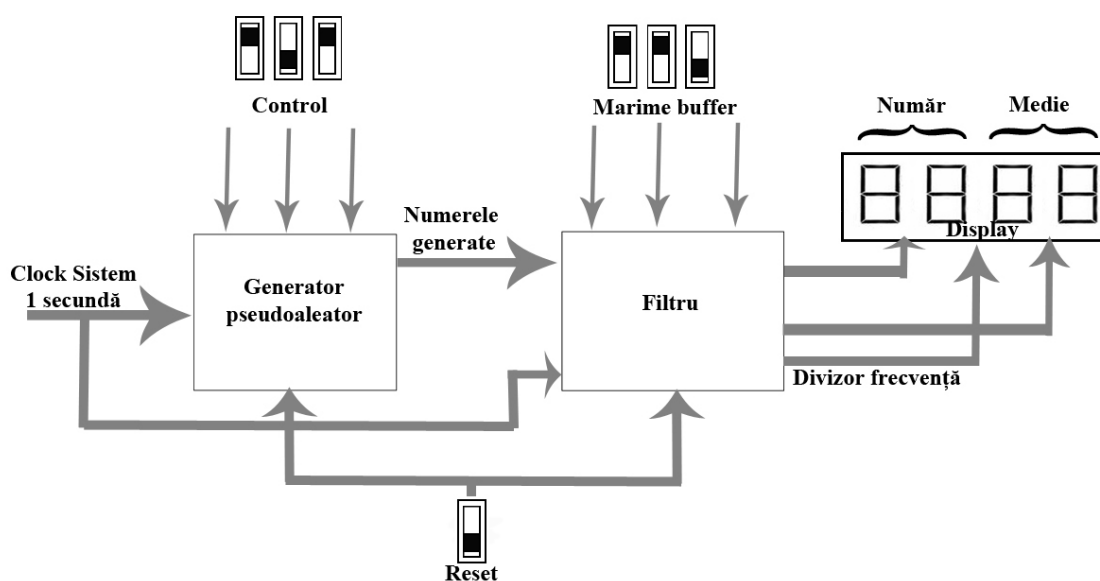
Control marime buffer

Off – Off – Off	Test => 0000-pe display
On – On – Off	Numere aleatoare între 0-15
On – On – On	Numere aleatoare între 0-255

Off – Off – Off	Stop – rămâne valoarea prezentă
On – Off – Off	Medie 2 numere
On – Off – On	Medie 4 numere
On – On – Off	Medie 8 numere
On – On – On	Medie 16 numere

Frecvență Clock = 1 secundă.

2.SCHEMA BLOC:



Componentele principale care alcătuiesc sistemul sunt: generatorul de numere pseudoaleatoare și filtrul. Pe lângă acestea 2 am mai putea include aici divizorul de frecvență realizat la o secundă (regăsit în schema bloc sub numele de *Clock Sistem 1 secundă*) cât și divizorul de frecvență folosit pentru a putea afișa pe display folosind toate cele 4 afișoare, conținând elemente diferite.

Interfața cu utilizatorul este realizată prin intermediul switch-urilor

Prin intermediul switch-urilor de control, utilizatorul poate să-și aleagă, folosind diferite combinații, următoarele opțiuni: Test; Numere aleatoare între 0-15; Numere aleatoare între 0-255.

Prin intermediul switch-urilor de control al bufferului, utilizatorul poate să-și aleagă, și aici, folosind tot diferite combinații, următoarele opțiuni: Stop – rămâne valoarea prezentă; Medie pentru 2 numere; Medie pentru 4 numere; Medie pentru 8 numere; Medie pentru 16 numere;

Resetarea sistemului se va face și ea cu ajutorul unui switch.

3.PROIECTARE:

Componentele folosite în realizarea sistemului de calcul sunt următoarele: generator de numere pseudoaleator, filtrul, divizor de frecvență, registru de deplasare, divizor de frecvență pentru anod. Funcționalitatea și realizarea acestor componente în VHDL va fi redată în continuare.

Generator de numere pseudoaleator

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;

--Iesirea Q intra in intrarea de la buffer
entity data_generator is
    port (
        Clk: in          std_logic;
        Control:in        std_logic_vector(2 downto 0);
        Q:                inout    std_logic_vector(7 downto 0)
    );
end entity;

--Functioneaza pe baza schemei din documentatie.
--Secventa: F, E, C, 8, 1, 2, 4, 9, 3, 6, D, A, 5, B, 7;  asta pentru
numere pe 4 biti
--Pe 8 biti secventa e prea lunga pt. a fi documentata. Sunt 64 de numere.
Incepe de la FF
architecture second_attempt of data_generator is
    signal Slow: std_logic;
```

```

component divizor_frecventa is
    port (
        CLK, Reset: in    STD_LOGIC;
        Q:                out STD_LOGIC
    );
end component;
begin
    div: divizor_frecventa port map (CLK => Clk, Reset => '1', Q =>
Slow);

    process(Slow, Control)
        variable D7, D6, D5, D4, D3, D2, D1, D0: std_logic := '1';
        variable DD3, DD2, DD1, DD0: std_logic := '1';
        variable Q7, Q6, Q5, Q4, Q3, Q2, Q1, Q0: std_logic := '0';
        variable QQ3, QQ2, QQ1, QQ0: std_logic := '0';
    begin
        if Control(2) = '1' and Control(1) = '1' and Control(0) = '0'
then --genereaza numere pe 4 biti
            --simuleaza bistabile! (de unde Q si D)
            if rising_edge(Slow) then
                QQ0 := DD0; QQ1 := DD1; QQ2 := DD2; QQ3 := DD3;

                DD0 := QQ2 xor QQ3;
                DD1 := QQ0; DD2 := QQ1; DD3 := QQ2;

            end if;
            Q(0) <= QQ0; Q(1) <= QQ1; Q(2) <= QQ2; Q(3) <= QQ3;
            Q(4) <= Q4; Q(5) <= Q5; Q(6) <= Q6; Q(7) <= Q7;

            -- 8 bits
            elsif Control(2) = '1' and Control(1) = '1' and Control(0) =
'1' then --genereaza numere pe 8 biti
                --simuleaza bistabile! (de unde Q si D)
                if rising_edge(Slow) then
                    Q0 := D0; Q1 := D1; Q2 := D2; Q3 := D3;
                    Q4 := D4; Q5 := D5; Q6 := D6; Q7 := D7;

                    D0 := Q6 xor Q7;
                    D1 := Q0; D2 := Q1;    D3 := Q2; D4 := Q3;
                    D5 := Q4; D6 := Q5; D7 := Q6;
                end if;
                Q(0) <= Q0; Q(1) <= Q1; Q(2) <= Q2; Q(3) <= Q3;
                Q(4) <= Q4; Q(5) <= Q5; Q(6) <= Q6; Q(7) <= Q7;

                elsif Control(2) = '0' and Control(1) = '0' and Control(0) =
'0' then -- Test Mode o/p 0 (Zero)
                    if Slow'Event and Slow = '1' then
                        Q(0) <= '0'; Q(1) <= '0'; Q(2) <= '0'; Q(3) <= '0';
                        Q(4) <= '0'; Q(5) <= '0'; Q(6) <= '0'; Q(7) <= '0';
                    end if;

                    else --daca combinatia de butoane nu este cunoscuta (Ex: 1, 0,
0)
                        --do nothing
                    end if;
                end process;
            end second_attempt;

```

Filtrul

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;

entity filter is
    port (
        Clk:          in std_logic;
        Reset:         in std_logic;
        Lungime:       in std_logic_vector(2 downto 0);
        Control:       in std_logic_vector(2 downto 0);
        Anodes:        out std_logic_vector(0 to 3);
        AfisorNumar:   out std_logic_vector(0 to 7)
    );
end filter;

--pana se umple bufferul afisam 0;
architecture first_attempt of filter is
    component data_generator is
        port (
            Clk: in std_logic;
            Control: in std_logic_vector(2 downto 0);
            Q: inout std_logic_vector(7 downto 0)
        );
    end component;

    component shift_register is
        port (
            Clk: in std_logic;
            iesire: out std_logic_vector(3 downto 0) := "1110"
        );
    end component;

    component divizor_frecventa is
        port (
            CLK: in std_logic;
            Reset: in std_logic;
            Q: out std_logic
        );
    end component;

    component divizor_anod is
        port (
            CLK, Reset: in std_logic;
            Q: out std_logic
        );
    end component;

    signal Data: std_logic_vector(7 downto 0);
    signal Suma: std_logic_vector(7 downto 0) := "00000000";
    signal AverageTemp: std_logic_vector(7 downto 0) := "00000000";
    signal Clk_slow: std_logic := '0';
    signal clk_anod: std_logic := '0';
    signal anod: std_logic_vector(3 downto 0) := "1110";

begin
    divizor_frecv: divizor_frecventa port map(
        CLK => clk,
        Reset => '1',
        Q => clk_slow
    );

```

```

N1: data_generator port map(
    CLK => Clk,
    Control => Control,
    Q => Data
);

anodss: divizor_anod port map(
    Clk => clk,
    Reset => '1',
    Q => clk_anod
);

shif: shift_register port map(
    Clk => Clk_anod,
    iesire => Anod
);

anodes <= anod;
AfisorNumar(7) <= '1';

process(Clk_slow, Lungime, Reset)
--depinde si de Lungime pentru ca atunci cand modificam dim 000 in
001 automat reseteaza datele
--generate pana atunci si genereaza altele, de lungimea potrivita
variable current: integer := 0;
type arr is array(0 to 15) of std_logic_vector(7 downto 0);
variable coada: arr := (others => "00000000");
begin
    if Reset = '1' then
        coada := (others => "00000000");
    else
        if Clk_slow'Event and Clk_slow = '1' then

            if Lungime = "000" then
                --hold data
                --adica afisam pe ecran Media
            elsif Lungime = "100" then --buffer = 2;

                if current = 0 and Data /= "00000000" then --
-> intarzie cu 1 tact, dar scapa de 0.
                    coada(0) := Data;
                    current := current + 1; --lungimea
coadei

                    elsif current > 0 then
                        coada(1) := coada(0); --al doilea
devine primul
                        coada(0) := Data; --primul devine
numarul citit

                        Suma <= coada(0) + coada(1);

                        current := current + 1;
                    end if;

                    if current > 1 then
                        AverageTemp <= SHR(Suma, "01");
                    end if;

                    elsif Lungime = "101" then -- buffer = 4

```



```

if current = 0 and Data /= "00000000" and Data
/= "UUUUUUUUU" then
    coada(0) := Data;
    current := current + 1;
elseif current > 0 then
    coada(3) := coada(2);
    coada(2) := coada(1);
    coada(1) := coada(0);
    coada(0) := Data;

    Suma <= coada(0) + coada(1) + coada(2) +
coada(3); --fara for, posibila eroare!!!

    current := current + 1;
end if;

if current > 3 then
    AverageTemp <= SHR(Suma, "10");
end if;

elseif Lungime = "110" then --buffer = 8

if current = 0 and Data /= "00000000" and Data
/= "UUUUUUUUU" then
    coada(0) := Data;
    current := current + 1;
elseif current > 0 then
    coada(7) := coada(6);
    coada(6) := coada(5);
    coada(5) := coada(4);
    coada(4) := coada(3);
    coada(3) := coada(2);
    coada(2) := coada(1);
    coada(1) := coada(0);
    coada(0) := Data;

    Suma <= coada(0) + coada(1) + coada(2) +
coada(3) +
coada(4) + coada(5) + coada(6) +
coada(7); --fara for, posibila eroare!!!

    current := current + 1;
end if;

if current > 7 then
    AverageTemp <= SHR(Suma, "0100");
end if;

elseif Lungime = "111" then

if current = 0 and Data /= "00000000" and Data
/= "UUUUUUUUU" then
    coada(0) := Data;
    current := current + 1;
elseif current > 0 then
    coada(15) := coada(14);
    coada(14) := coada(13);
    coada(13) := coada(12);
    coada(12) := coada(11);
    coada(11) := coada(10);
    coada(10) := coada(9);

```

```

        coada(9) := coada(8);
        coada(8) := coada(7);
        coada(7) := coada(6);
        coada(6) := coada(5);
        coada(5) := coada(4);
        coada(4) := coada(3);
        coada(3) := coada(2);
        coada(2) := coada(1);
        coada(1) := coada(0);
        coada(0) := Data;

        Suma <= coada(0) + coada(1) + coada(2) +
coada(3) + coada(4) +
coada(5) + coada(6) + coada(7) +
coada(8) + coada(9) + coada(10) +
coada(11) + coada(12) + coada(13) +
coada(14);

        current := current + 1;
    end if;

    if current > 15 then
        AverageTemp <= SHR(Suma, "1000");
    end if;

end if;

end if;

end process;

process(Clk_anod)
variable stare: integer := 0;
begin
    if anod = "1110" then
        case AverageTemp(3 downto 0) is
            when "0000" => AfisorNumar(0 to 6) <= "0000001";
            when "0001" => AfisorNumar(0 to 6) <= "1001111";
            when "0010" => AfisorNumar(0 to 6) <= "0010010";
            when "0011" => AfisorNumar(0 to 6) <= "0000110";
            when "0100" => AfisorNumar(0 to 6) <= "1001100";
            when "0101" => AfisorNumar(0 to 6) <= "0100100";
            when "0110" => AfisorNumar(0 to 6) <= "0100000";
            when "0111" => AfisorNumar(0 to 6) <= "0001111";
            when "1000" => AfisorNumar(0 to 6) <= "0000000";
            when "1001" => AfisorNumar(0 to 6) <= "0000100";
            when "1010" => AfisorNumar(0 to 6) <= "0001000"; --
10 - A
            when "1011" => AfisorNumar(0 to 6) <= "1100000"; --
11 - b
            when "1100" => AfisorNumar(0 to 6) <= "0110001"; --
12 - C
            when "1101" => AfisorNumar(0 to 6) <= "1000010"; --
13 - d
            when "1110" => AfisorNumar(0 to 6) <= "0110000"; --
14 - E
            when "1111" => AfisorNumar(0 to 6) <= "0111000"; --
15 - F

```

```

        when others => AfisorNumar(0 to 6) <= "1000001"; --
in caz de stare necunoscuta -> U
    end case;
    elsif anod = "1101" then
        case AverageTemp(7 downto 4) is
            when "0000" => AfisorNumar(0 to 6) <= "0000001";
            when "0001" => AfisorNumar(0 to 6) <= "1001111";
            when "0010" => AfisorNumar(0 to 6) <= "0010010";
            when "0011" => AfisorNumar(0 to 6) <= "0000110";
            when "0100" => AfisorNumar(0 to 6) <= "1001100";
            when "0101" => AfisorNumar(0 to 6) <= "0100100";
            when "0110" => AfisorNumar(0 to 6) <= "0100000";
            when "0111" => AfisorNumar(0 to 6) <= "0001111";
            when "1000" => AfisorNumar(0 to 6) <= "0000000";
            when "1001" => AfisorNumar(0 to 6) <= "0000100";
            when "1010" => AfisorNumar(0 to 6) <= "0001000"; --
10 - A
            when "1011" => AfisorNumar(0 to 6) <= "1100000"; --
11 - b
            when "1100" => AfisorNumar(0 to 6) <= "0110001"; --
12 - C
            when "1101" => AfisorNumar(0 to 6) <= "1000010"; --
13 - d
            when "1110" => AfisorNumar(0 to 6) <= "0110000"; --
14 - E
            when "1111" => AfisorNumar(0 to 6) <= "0111000"; --
15 - F
            when others => AfisorNumar(0 to 6) <= "1000001"; --
in caz de stare necunoscuta -> U
    end case;
    elsif anod = "1011" then
        case Data(3 downto 0) is
            when "0000" => AfisorNumar(0 to 6) <= "0000001";
            when "0001" => AfisorNumar(0 to 6) <= "1001111";
            when "0010" => AfisorNumar(0 to 6) <= "0010010";
            when "0011" => AfisorNumar(0 to 6) <= "0000110";
            when "0100" => AfisorNumar(0 to 6) <= "1001100";
            when "0101" => AfisorNumar(0 to 6) <= "0100100";
            when "0110" => AfisorNumar(0 to 6) <= "0100000";
            when "0111" => AfisorNumar(0 to 6) <= "0001111";
            when "1000" => AfisorNumar(0 to 6) <= "0000000";
            when "1001" => AfisorNumar(0 to 6) <= "0000100";
            when "1010" => AfisorNumar(0 to 6) <= "0001000"; --
10 - A
            when "1011" => AfisorNumar(0 to 6) <= "1100000"; --
11 - b
            when "1100" => AfisorNumar(0 to 6) <= "0110001"; --
12 - C
            when "1101" => AfisorNumar(0 to 6) <= "1000010"; --
13 - d
            when "1110" => AfisorNumar(0 to 6) <= "0110000"; --
14 - E
            when "1111" => AfisorNumar(0 to 6) <= "0111000"; --
15 - F
            when others => AfisorNumar(0 to 6) <= "1000001"; --
in caz de stare necunoscuta -> U
    end case;
    elsif anod = "0111" then
        case Data(7 downto 4) is
            when "0000" => AfisorNumar(0 to 6) <= "0000001";
            when "0001" => AfisorNumar(0 to 6) <= "1001111";

```

```

when "0010" => AfisorNumar(0 to 6) <= "0010010";
when "0011" => AfisorNumar(0 to 6) <= "0000110";
when "0100" => AfisorNumar(0 to 6) <= "1001100";
when "0101" => AfisorNumar(0 to 6) <= "0100100";
when "0110" => AfisorNumar(0 to 6) <= "0100000";
when "0111" => AfisorNumar(0 to 6) <= "0001111";
when "1000" => AfisorNumar(0 to 6) <= "0000000";
when "1001" => AfisorNumar(0 to 6) <= "0000100";
when "1010" => AfisorNumar(0 to 6) <= "0001000"; --
10 - A
when "1011" => AfisorNumar(0 to 6) <= "1100000"; --
11 - b
when "1100" => AfisorNumar(0 to 6) <= "0110001"; --
12 - C
when "1101" => AfisorNumar(0 to 6) <= "1000010"; --
13 - d
when "1110" => AfisorNumar(0 to 6) <= "0110000"; --
14 - E
when "1111" => AfisorNumar(0 to 6) <= "0111000"; --
15 - F
when others => AfisorNumar(0 to 6) <= "1000001"; --
in caz de stare necunoscuta -> U
end case;
else
    AfisorNumar(0 to 6) <= "1000001";
end if;
end process;

end first_attempt;

```

Divizor de frecvență

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity divizor_frecventa is
    port(
        CLK,Reset: in STD_LOGIC;
        Q: out STD_LOGIC
    );
end entity;

architecture flash of divizor_frecventa is
begin
    process(CLK,Reset)
        variable intQ : STD_LOGIC_VECTOR(24 downto 0) := (others => '0');
        variable x: STD_LOGIC := '0';
    begin
        if Reset='1' then
            if (CLK'EVENT) and (CLK='1') then
                intQ := intQ + 1;
                if intQ = "10111110101111000010000000" then -- 25000000 clocks to
change output
                    Q <= x;
                    intQ := (others => '0');
                    x := not x;
                end if;
            end if;
        end if;
    end process;
end architecture;

```

```

        end if;
    else
        Q <= '0';
    end if;
end process;
end flash;

```

Registru de deplasare

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;
use IEEE.std_logic_unsigned.all;

entity shift_register is
    port (
        Clk: in    std_logic;
        iesire: out std_logic_vector(3 downto 0)
    );
end shift_register;

architecture arh of shift_register is
    signal X: std_logic_vector(3 downto 0) := "1110";
begin
    process(Clk)
    begin
        if rising_edge(Clk) then
            case X is
                when "1110" =>
                    iesire <= "1101";
                    X <= "1101";
                when "1101" =>
                    iesire <= "1011";
                    X <= "1011";
                when "1011" =>
                    iesire <= "0111";
                    X <= "0111";
                when others =>
                    iesire <= "1110";
                    X <= "1110";
            end case;
        end if;
    end process;
end architecture arh;

```

Divizor de frecvență pentru anod

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity divizor_anod is
    port(
        CLK, Reset: in    STD_LOGIC;
        Q:          out STD_LOGIC
    );
end entity;

```

```

architecture flash of divizor_anod is
begin
  process(CLK,Reset)
    variable intQ : STD_LOGIC_VECTOR(14 downto 0) := (others => '0');
    variable x: STD_LOGIC := '0';
  begin
    if Reset='1' then
      if (CLK'event) and (CLK ='1') then
        intQ := intQ + 1;
        if intQ >= "110000110101000" then -- 25000 clocks to change output
(1ms period)
          Q <= x;
          intQ := (others => '0');
          x := not x;
        end if;
      end if;
    else
      Q <= '0';
    end if;
  end process;
end flash;

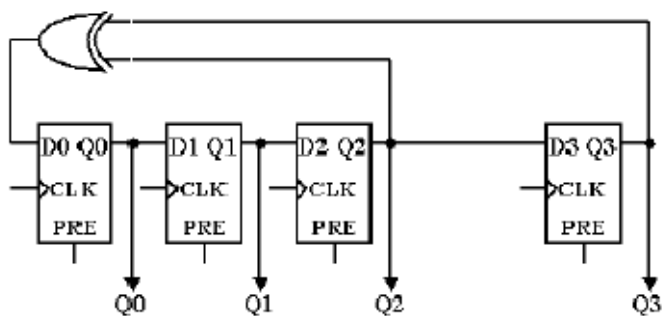
```

4.INTRĂRI ȘI IEȘIRI. SEMNALE INTERNE:

Pentru **generatorul de numere pseudoaleatoare** am folosit următoarele semnale: ca și semnale de intrare sunt Clk, control, de intrare-ieșire Q. Pe lângă acestea s-a mai folosit și un semnal intern Slow care va recepționa semnalul venit pe ieșirea divizorului de frecvență regăsit în proiect sub denumirea de *divizor_frecvență*.

Semnalul Clk este folosit asemenea unui semnal de tact. Control este un vector de 3 elemente folosit cu scopul de exprima switch-urile de pe plăcuță, cele de control. Q reprezintă ieșirea pe 7 biți a generatorului de numere pseudoaleatoare care va deveni mai târziu intrare pentru componenta *filter*.

În interiorul acestei componente am folosit anuite variabile identice cu intrările și ieșirile uni bistabil D, circuitul care a stat la baza creării acestei componente care generează numerea pseudoaleatoare ce sunt vizibile pe display-ul plăcuței la intervalul de timp de o secundă datorită folosirii divizoului de frecvență. Soluția aleasă de generare a numerelor este ilustrată prin intermediul imaginii de mai jos.



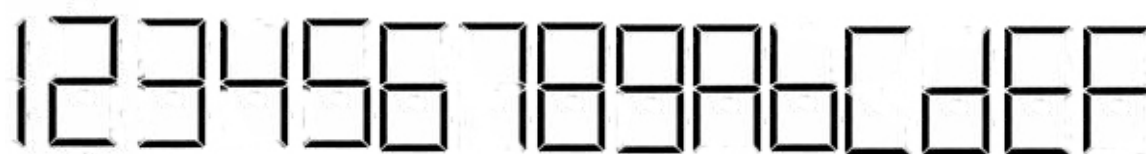
Pentru **divizorul de frecvență** am folosit următoarele semnale: Clk și Reset semnale de intrare, iar ca semnal de ieșire Q. Clk semnifică semnalul de tact, Reset-ul așa cum îi spune și numele are rolul de a reseta divizorul, iar Q este ieșirea activată la sfârșitul procesului de numărare. Această componentă este folosită cu scopul de a ține ocupat procesorul plăcuței ”dându-i de lucru” să numere aducând astfel procesul de apariție a numerelor și implicit a clock-ului sistemului la o secundă.

Pentru **divizor anod** am folosit semnalele ca și cele de la divizorul de frecvență și anume: Clk și Reset semnale de intrare, iar semnal de ieșire Q. Clk semnifică semnalul de tact, Reset-ul așa cum îi spune și numele are rolul de a reseta divizorul, iar Q este ieșirea activată la sfârșitul procesului de numărare. Această componentă este folosită cu scopul de a ține ocupat procesorul plăcuței ”dându-i de lucru” să numere aducând astfel procesul de apariție a numerelor pe display se va întâmpla în ”timp real”.

Pentru **registru de deplasare** am folosit ca și semnal de intrat Clk, iar ca ieșire un semnal denumit chiar ieșire. Clk după cum semnifică și denumirea sa reprezintă semnalul de tact aflat în lista de sensibilitate a procesului din care face parte. Pe semnalul ieșire se va trimite rezultat după deplasare. Pe lângă aceste două semnale am mai folosit un semnal intern cu rolul de a reține starea anterioară în care s-a aflat registrul. Această componentă are rolul de a simula apariția ”simultană” numerele pe display, cum folosirea celor 4 afișoare simultană este imposibilă acest lucru se face posibil prin intermediul acestei componente. Totul se întâmplă la o frecvență foarte mare astfel încât nu se observă această deplasare care se întâmplă foarte rapid.

Pentru **filtru**, componenta în care se întâmplă cele mai importante lucruri, bineînțeles și cele mai multe, am folosit ca semnale de intrare Clk, Reset, Lungime și Control, iar ca semnale de ieșire Anodes și AfiosrNumar. Clk reprezintă semnalul de tact creat prin intermediul *divizorului de frecvență* la o secundă, iar Reste este elementul care va reseta tot sistemul readucându-l la valoarea inițială de 0. Lungime are rolul de a primi din datele introduse de utilizator, prin intermediul switch-urilor ungimea de numere ce se va genera,

implicit mărimea alocată buffer-ului. Control reprezintă semnalul primit de pe switch-urile puse la dispoziție utilizatorului pentru diferitele opțiuni care-i sunt la dispoziție și au fost prezentate în primele pagini ale documentației. Anodes este semnalul care selectează anodul pe care se va afișa numărul. Numerele se vor afișa în hexazecimal, dar înainte de a fi afișate se va face legătura la componenta *shift_register*, după cum am prezentat-o mai sus cu scopul de a vedea numerele de pe display în timp real chiar dacă acest lucru nu se poate întâmpla simultan pe toate cele 4 afișoare. AfișorNumar reprezintă un vector care definește cele 7 segmente care alcătuiesc un afișor, ele au semnificația conform imaginii de mai jos.



Ca și semnale interne am folosit: Data, Suma, AverageTemp, Clk_slow, clk_anod, anod. Data reprezintă numărul venit de pe ieșirea *generatorului pseudoaleator*, Suma reține suma până în momentul de față, după cum îi spune și numele, AverageTemp este un semnal care reține temporar media, Clk_slow reprezintă tactul sistemului realizat prin componenta *divizor de frecvență* la o secundă, clk_anod reprezintă tactul realizat pentru anod regăsit în componenta *divizor anod*, iar anod reprezintă anodul selectat unde se va face afișarea numărului, acest lucru se va întâmpla și prin intermediul componentei *shift_register*.

5.JUSTIFICAREA SOLUȚIEI ALESE:

Pentru implementarea proiectului am folosit 5 componente cu scopul de a ilustra mai bine funcționalitatea sistemului de calcul. Cele 5 componente au fost prezentate mai sus, iar schema de legătură dintre componente poate fi urmărită prin intermediul schemei bloc prezentată în primele pagini.

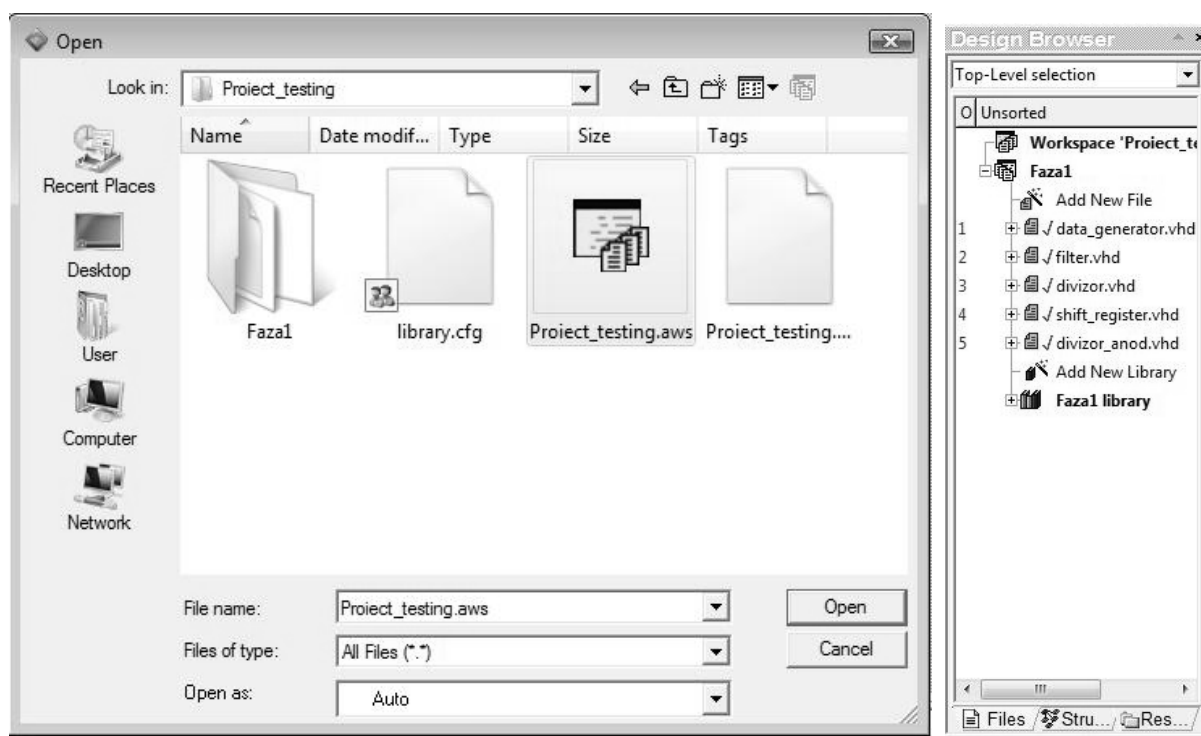
Datorită acestei soluții legarea componentelor s-a realizat relativ ușor, iar urmărirea metodei de către un utilizator mai mult sau mai puțin experimentat va fi deopotrivă mai ușoară atât de vizualizat ca metodă cât și de înțeles ca și cod sursă.

În cadrul soluției alese procesele și lista de sensibilitate a acestora joacă un rol important în definirea sistemului de calcul. Unirea tuturor componentelor realizate s-a efectuat în interiorul procesului componentei *filter*.

6.INSTRUCTIUNI DE UTILIZARE – MANUALUL UTILIZATORULUI:

1.Active HDL 7.2

Se rulează programul Active-HDL 7.2. După ce acesta s-a încărcat alegem din meniul File opțiunea Open, iar pe ecran se va deschide o fereastră unde avem posibilitatea de a selecta proiectul din directorul unde l-am salvat. În exemplul prezentat mai jos proiectul se regăsește în directorul "Proiect_testing", de unde alegem să deschidem "Proiect_testing.aws".



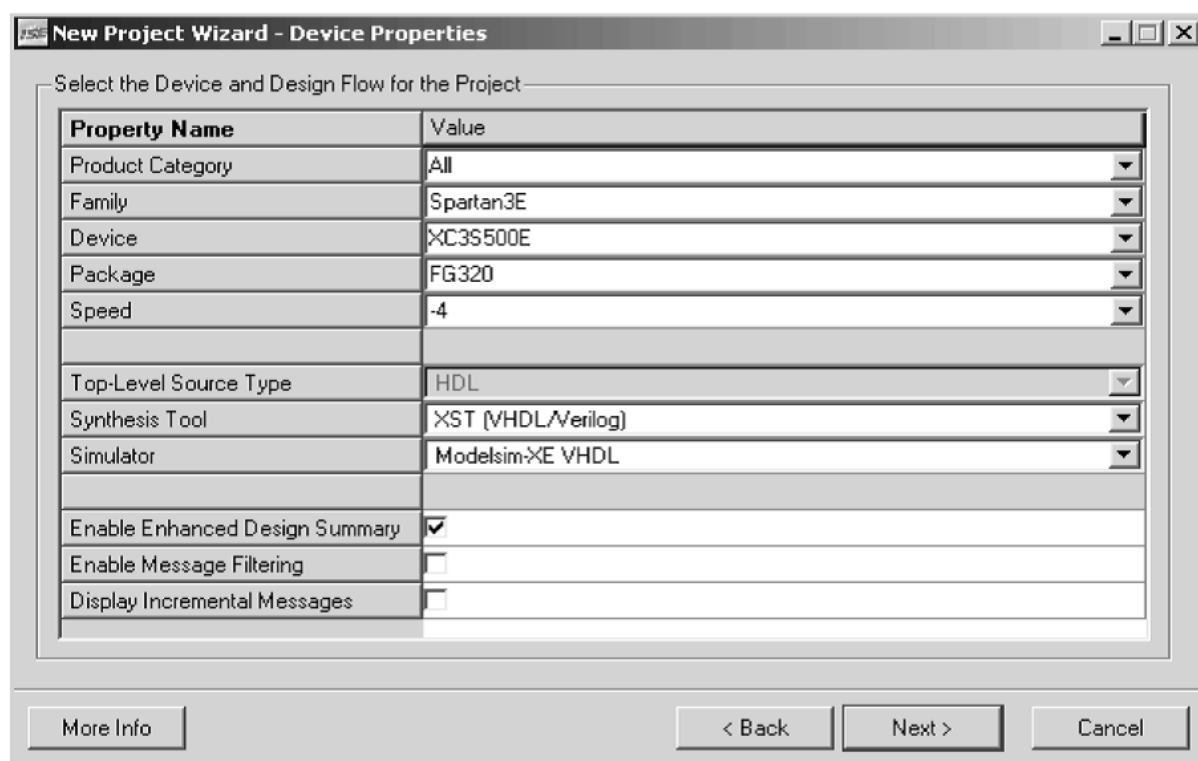
În meniul din partea stângă a ecranului "Design Browser" se pot vedea toate elementele ce compun proiectul sub denumirea dată de utilizator. În cazul nostru avem: data_generator, filter, divizor, shift_register și divizor_anod, toate cu extensia .vhd. Selectând unul din aceste componente Active-HDL va deschide o fereastră în partea dreapta unde se va putea vedea codul în vederea de citire sau modificare.

La acest proiect o eventuală vizualizare a lui în simulatorul programului Active-HDL va fi imposibilă datorită prezenței ca și componentă vitală a funcționării în timp real pe plăcuța FPGA a unui divizor de frecvență.

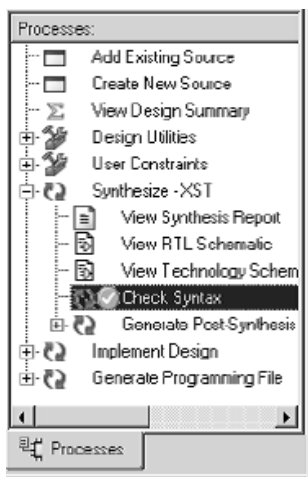
2.Xilinx ISE și plăcuța FPGA

Se va rula programul Xilinx ISE, apoi se va alege opțiunea de proiect nou astfel: File – New Project. Utilizatorul va putea denumi noul proiect după bunul plac, continuând apoi

apăsând butonul Next. În continuare se va alege tipul plăcuței utilizate, asemănător exemplului din imaginea de mai jos:



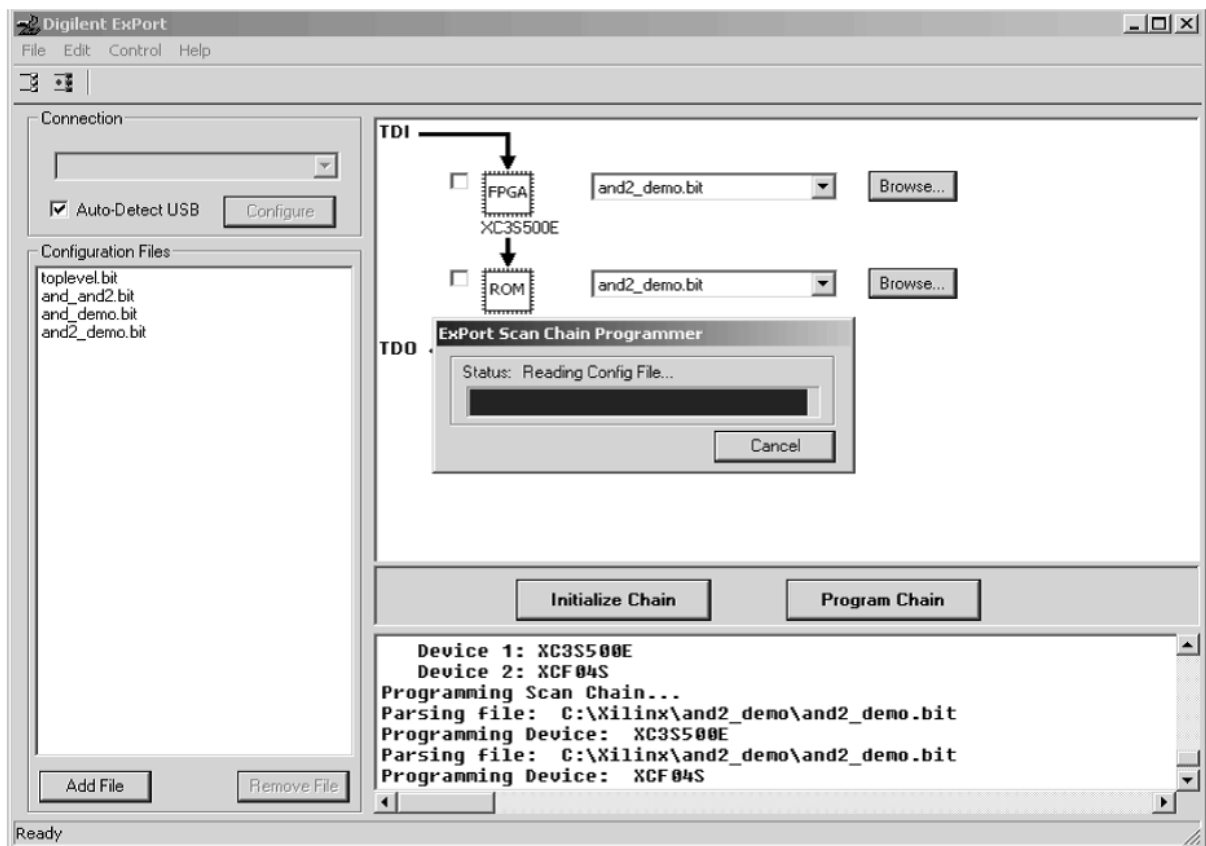
și după această opțiune se va continua prin apăsarea butonului Next apoi Finish.



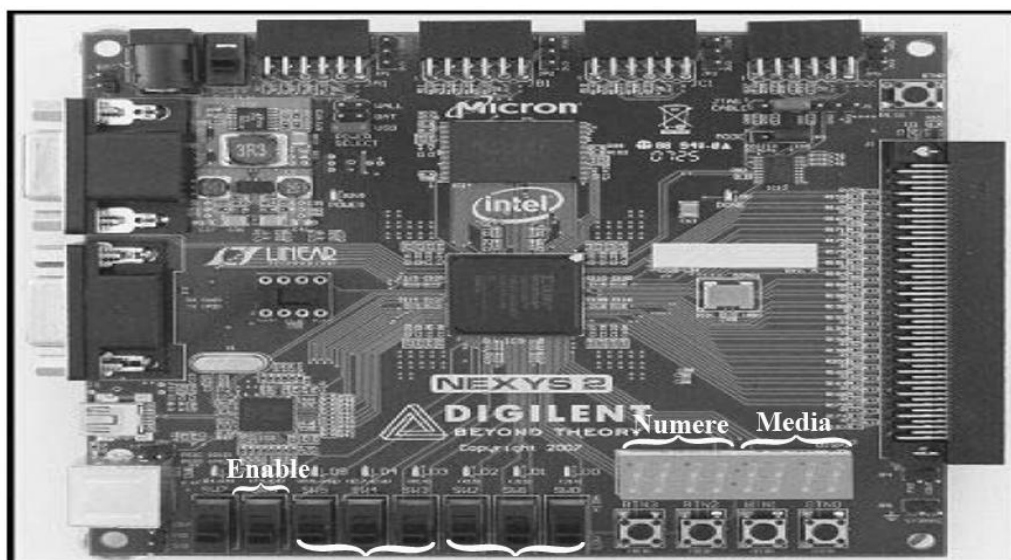
În continuarea utilizatorul va trebui să aleagă sursa folosind opțiunea Add Source de unde va selecta toate fișierele sursă regăsite cu extensi .src. Acestea se vor găsi în cazul nostru la adresa următoare: Project_testing/Faza1/Src/... Astfel codul realizat în Vhdl este deschis prin intermediul programului Xilinx. În partea stângă a ecranului din fereastra Processes se va alege opțiunea User Constraints – Assign Package Pins pentru selectarea pinilor de intrare respectiv ieșire respectând denumirea pinilor de pe plăcuța folosită. După această operațiune se va selecta Generate

Programming File.

După toate aceste procese pregătitoare proiectul este pregătit să fie încărcat pe plăcuță. Se rulează programul Digilent ExPort, primul pas se inițializează plăcuța prin opțiunea "Initialize Chain" apoi se alege programul ce se dorește a fi încărcat pe plăcuță cu extensia .bit. După care se va apăsa butonul Program Chain care va deveni activ. Exemplul prezentat în imaginea de mai jos este pentru un proiect cu denumirea de: and2_demo.



În acest moment pe plăcuță a fost încărcat programul realizat în Active-HDL și nu ne rămâne decât să îl testăm "real" cu mâna noastră prin intermediul plăcuței FPGA.



Panou de control Control Bufer

7.POSIBILITĂȚI DE DEZVOLTARE ULTERIOARĂ. ÎMBUNĂTĂȚIRI:

O primă îmbunătățire a sistemului de calcul prezentat ar putea fi o aproximare exactă a numerelor, astfel: cele care au zecimalele mai mare sau egale cu 50 să fie approximate prin

adaos la numărul următor, iar cele mai mici de 50 sa fie approximate prin lipsă la numărul precedent. Astfel exactitatea mediei numerelor generate va crește.

Altă îmbunătățire ce ar putea prinde contur în viitor ar putea fi aceea de a facilita utilizatorul printr-o gamă largă de opțiuni. Una dintre ele ar putea fi de a selecta numere mai mari de 8 biți. Astfel am putea extinde sistemul de calcul și pentru numere mai mari de 255, crescându-i aplicabilitatea.